



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	KARL GABRIEL R. JARILLA
Section:	AN42
Professor:	MS. CRISOLA TAN
Date:	October 3, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
(Example: TA1_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
```

```
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
 - **Number of agents** (e.g., 5)
 - **Grid size** (e.g., 10 for a 10x10 grid)
 - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
 - How agents spread across the grid over time
 - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
 - **More agents** (e.g., 10 or 20)
 - **Smaller/larger grid sizes**
 - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.

2. **Change movement rules:** Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions:** Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios:** Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ↴

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import collections

# ----- Get user input from console -----
print("--- Random Walk Simulation (Laboratory Tasks) ---")
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

print("\n--- Select Movement Rule (Laboratory Task 2 & 4) ---")
print("1. Standard Random Walk")
print("2. Biased Walk (Prefers moving Right)")
print("3. Avoidance (Agents avoid stepping on occupied cells)")
rule_choice = input("Enter rule number (1-3): ")

movement_rule = 'random'
if rule_choice == '2':
    movement_rule = 'biased'
elif rule_choice == '3':
    movement_rule = 'avoid'

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def setup(self):
        # Laboratory Task 1: Initialize path tracking
        self.history_x = []
        self.history_y = []

    def record_position(self):
        # Laboratory Task 1: Save current position to path
        self.history_x.append(self.position[0])
        self.history_y.append(self.position[1])

    def step(self):
```

```

x, y = self.position

# Laboratory Task 2: Change movement rules
if self.model.p.movement_rule == 'biased':
    # Bias direction: 40% right, 20% left, 20% up, 20% down
    direction = random.choices(
        [(1,0), (-1,0), (0,1), (0,-1)],
        weights=[0.4, 0.2, 0.2, 0.2],
        k=1
    )[0]
else:
    # Standard random choice
    direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])

# Keep agents inside grid
new_x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
new_y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))

# Laboratory Task 2: Avoidance Rule
if self.model.p.movement_rule == 'avoid':
    # Check if the target cell is already occupied by another agent
    occupied = any(a.position == (new_x, new_y) for a in self.model.agents if a != self)
    if not occupied:
        self.position = (new_x, new_y)
    # If occupied, the agent simply waits in place for this step
else:
    self.position = (new_x, new_y)

self.record_position()

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        # For avoidance rule, ensure they don't spawn on the same cell if possible
        if self.p.movement_rule == 'avoid' and self.p.agents <= (self.p.grid_size[0] * self.p.grid_size[1]):
            all_cells = [(i, j) for i in range(self.p.grid_size[0]) for j in range(self.p.grid_size[1])]
            start_positions = random.sample(all_cells, self.p.agents)
            for agent, pos in zip(self.agents, start_positions):
                agent.position = pos
        else:
            for agent in self.agents:

```

```

        agent.position = (random.randint(0, self.p.grid_size[0]-1),
                          random.randint(0, self.p.grid_size[1]-1))

    # Record the starting position for the path
    for agent in self.agents:
        agent.record_position()

    def step(self):
        # Shuffle agents to prevent turn-order bias when avoiding each other
        agents_shuffled = list(self.agents)
        random.shuffle(agents_shuffled)
        for agent in agents_shuffled:
            agent.step()

# ----- Parameters -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps,
    'movement_rule': movement_rule
}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots(figsize=(6, 6))
ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)
ax.set_xticks(range(model.p.grid_size[0]))
ax.set_yticks(range(model.p.grid_size[1]))
ax.grid(True, linestyle='--', alpha=0.5)
ax.set_title(f"Random Walk Simulation\nRule: {movement_rule.capitalize()}")

# Scatter plot for current positions
scat = ax.scatter([], [], s=150, zorder=5, color='red')

# Laboratory Task 1: Line objects for tracking paths
lines = [ax.plot([], [], '-', alpha=0.4, linewidth=2)[0] for _ in range(num_agents)]

def update(frame):
    model.step()

```

```

# Update current positions
x = [agent.position[0] for agent in model.agents]
y = [agent.position[1] for agent in model.agents]
scat.set_offsets(list(zip(x, y)))

# Update path trails
for i, agent in enumerate(model.agents):
    lines[i].set_data(agent.history_x, agent.history_y)

return [scat] + lines

# Run animation
ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()

# ----- Laboratory Task 3: Analyze Final Positions -----
# Note: This executes in the console after the animation window is closed.
print("\n--- Final Analysis ---")
final_positions = [agent.position for agent in model.agents]
print("Final Agent Positions:", final_positions)

# Group and count distributions
distribution = collections.Counter(final_positions)
print("\nPosition Distribution (Cells with agents):")
for pos, count in distribution.items():
    print(f"Cell {pos}: {count} agent(s)")

# Analyze by Quadrant to observe clustering
mid_x, mid_y = grid_size / 2, grid_size / 2
quadrants = {'Top-Right': 0, 'Top-Left': 0, 'Bottom-Left': 0, 'Bottom-Right': 0}

for (x, y) in final_positions:
    if x >= mid_x and y >= mid_y: quadrants['Top-Right'] += 1
    elif x < mid_x and y >= mid_y: quadrants['Top-Left'] += 1
    elif x < mid_x and y < mid_y: quadrants['Bottom-Left'] += 1
    else: quadrants['Bottom-Right'] += 1

print("\nAgent Distribution by Quadrant:")
for quad, count in quadrants.items():
    print(f"{quad}: {count} agent(s)")

```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
- 5.

Answer the Questions

1. The grid becomes densely populated, and the outcome depends on the specific movement rules. In a standard random walk, multiple agents will frequently occupy the same cell. If an avoidance rule is implemented (where agents cannot share cells), heavy congestion occurs, agents are frequently forced to skip turns, and the system may experience complete gridlock if the number of agents exceeds the total number of cells.
2. Increasing the number of steps allows agents to travel further from their starting positions, expanding their overall spread. Over a long enough duration within a bounded grid, the agents will eventually reach the boundaries, bounce back, and settle into a roughly uniform distribution across the entire available space.
3. No, the agents never move outside the grid. This is because the simulation code explicitly restricts their coordinates using bounding functions (e.g., `min()` and `max()`). If a random step would take an agent beyond the grid's minimum (0) or maximum dimensions, the code forces their position to remain exactly at the edge boundary.
4. By modifying its rules and environment, this simulation framework can model a wide variety of real-life complex systems. In physics and chemistry, a standard random walk perfectly models the Brownian motion of diffusion, such as gas particles spreading across a room or dye dispersing in water. For crowd dynamics, introducing avoidance rules mimics how humans navigate dense spaces like subway stations, while adding directional biases can simulate pedestrians flowing toward an exit during an evacuation. Additionally, by assigning a health status to the agents, the simulation can model epidemiological disease transmission as infected and susceptible individuals interact in the same space. Finally, in biology, it can simulate foraging behaviors or chemotaxis, representing how organisms navigate randomly but with a slight bias toward higher concentrations of nutrients.